

SRW3 — Phase 1D-R1-FINAL: Evidence and Provenance Closure

Phase: 1D-R1-FINAL (closure of Phase 1D-R1; Phase 1E remains closed until this closure is complete)

Status: ALL CLOSURE CONDITIONS MET — see §12 (completion criterion).

Predecessor: SRW3-Phase1D-R1-Verifier-Audit-Report (the soundness-fix audit; terminology and citations updated in place, audited content unchanged).

1. Executive summary

Phase 1D-R1 was scientifically complete at the verifier level when this session began, but archival closure was blocked by two evidence/provenance defects and one manifest defect. First, the transcript `part7_k_composition_suite.txt` recorded a K toolchain-loader failure, while the companion note `part7_k_composition_note.txt` reported a successful K-side composition result that the transcript therefore did not evidence: a claim without its verbatim proof. Second, the git bundle `git/srw3-work.bundle` terminated at `phase1-start @ 80448f2` and contained none of the R1 commits, so the shipped provenance could not reconstruct the audited artifact. Third, the artifact `MANIFEST.txt` checksummed itself from inside itself, a dangling self-reference that could never verify.

This closure run resolves all three defects without deleting a single historical evidence file, re-executes the full regression and adversarial program on a live-rebuilt toolchain, and freezes the final state in a descendant commit tagged `phase1d-r1-complete`, a verified fresh-repository git bundle, and a portable artifact bundle whose manifest excludes itself from its own checksum. The scientific result being frozen is unchanged from the R1 audit and is restated with its exact scope in §12: within the modeled verifier domain, a validly authorized but dishonest producer cannot make an internally inconsistent presented post-state pass the verifier's state-composition layer after the L7 repair. Historically true execution effects remain an open problem requiring authenticated execution/state evidence and are explicitly not claimed here.

2. Preclosure state freeze (mandate Part 1)

Before any closure action, the arriving state was frozen verbatim in `transcripts/audit/phase1d_r1/part0_final_freeze.txt`. The working tree was `phase1c @ e828643` with a clean status; the committed tree object hashes to `346d95ed0ce4033a2aa6601e2960f7e100c20ade` and the git archive of that tree hashes to `dc674810ca9c5cacbe8b8e39ff7c1b92ecacf1983904215523102d8219624c7c`. The shipped artifact bundle `SRW3-Phase1D-R1-Artifact-Bundle.zip` (2,812,074 bytes, SHA-256 `b54d896812cd4811bd92af0442a4cc149a04ca6eb98e023616a553e6e205724e`) was copied unchanged to `SRW3-Phase1D-R1-preclosure.zip` with a matching sidecar, so the preclosure evidence remains obtainable forever at its exact pre-closure bytes.

The toolchain was rebuilt from the pinned recipe before the freeze was completed, because the sandbox had been reset between sessions: K v7.1.337 (jammy deb, extracted without root), z3 4.13.3, the clang/LLVM-15 chain at 15.0.6-4+b1 (bookworm pool, including `libclang-common-15-dev` which supplies the shim build's `stddef.h` — a recovery gap recorded into `scripts/rebuild_env_1d.sh` during this run), `libsecp256k1` 0.5.0 with the recorded soname shims, KEVM v1.0.921 source, and `blockchain-k-plugin` K sources at pinned SHA `207ae512`. The environment script was reconstructed at `tools/env.sh` following the recorded recipe, and the `llvm-kompile-clang` driver was re-patched (original preserved as `.orig`) to honor `LLVM_KOMPILE_CXX`.

Every prior transcript was preserved, and this deserves an explicit sentence because the mandate singles it out: the stale failed K transcript `part7_k_composition_suite.txt` and the aspirational note `part7_k_composition_note.txt` are both retained exactly as they arrived, and neither file was modified, renamed, or deleted at any point in this closure.

3. Closure blocker 1 resolved: the K composition transcript (mandate Part 2)

Root-causing the failed transcript took one step: the frozen error text shows `kore-expand-macros` failing to load `libLLVM-15.so.1`, which is a loader-path defect of the session environment, not a semantic defect of the definition. After the pinned-recipe rebuild, the identical command succeeded. The suite represented by `semantics/phase1d/srw3lin-r1.k` and `semantics/phase1d/srw3lin-r1-demo.k` (repository paths `k/phase1d/srw3lin-r1.k`, `k/phase1d/srw3lin-r1-demo.k`) was then executed in two independent ways, and both were captured verbatim in the NEW transcript `transcripts/audit/phase1d_r1/part7_k_composition_suite_final.txt`.

Run (A) rebuilds everything from source: the `krypto shim` is compiled from `k/phase1d/shim/` with the pinned `clang-15` (the one-line `keccak-multiblock` fix present, marker-checked), `srw3lin-r1-demo.k` is re-komplied with `--backend llvm --hook-namespaces KRYPTO` into a fresh `r1-out-final`, and the suite is run there. Run (B) executes the committed compiled definition `k/phase1d/r1-out` exactly as committed at the R1 head. Both runs exit 0 and both print the same final four verdicts:

```
pos=valid;neg1=invalid-state-composition;neg2=invalid-state-composition;neg3=invalid-state-composition
```

The transcript records the exact commands, the backend (`llvm`, per `r1-out/backend.txt`), the toolchain versions, the `kompile/krun` exit codes, and the verbatim `krun` outputs. The report's §8, §13 and the §17 artifact index now cite the FINAL transcript; the historical failed run remains in place beside it, and the note that reported an unevidenced success is likewise preserved so the provenance gap itself stays auditable.

4. The K-side exact composition semantics (mandate Part 3)

The final evidence establishes `PresentedPost == Apply(PreState, TrueEffects)` through the K rule

```
rule LinCompositionOk(CTX:LinCtx)
=> BytesEq( LinCanonKV(linCtxPostP(CTX)),
LinCanonKV(LinApplyEff(linCtxPre(CTX), linCtxEffP(CTX))) )
```

which is the byte-level form of canonical finite-map equality. Symmetry holds because `BytesEq(B, B) => true` with `BytesEq(B1, B2) => false [owise]` compares its two arguments symmetrically, and the two arguments are exactly the canonical encodings of the presented post-state and of the composition of the claimed pre-state with the true effects. Canonicity holds because `LinCanonKV(M) = I2B4(sizeMap(M)) + LinKVIter(M)` extracts accounts by `MinKeyOfMap` in ascending order and slots likewise, so every finite map has exactly one byte string; the encoding is injective and order-deterministic rather than positional.

The three rejection classes follow structurally and are demonstrated concretely by the suite. An extra key changes the size prefix and appends an ascending-key section, so the byte strings differ (`neg1`, the L7 extra-key attack with entry `(2,9):=999`); a missing key shortens the encoding and removes a section (`neg2`); a changed value alters the slot's value bytes at identical key layout (`neg3`). The semantics were not weakened to make the demonstration pass: run (A) komplied the committed source bytes, and the source files' SHA-256 hashes are recorded in the final transcript beside the verdicts.

5. Final regression matrix (mandate Part 7)

The complete regression program was re-executed on the rebuilt toolchain, live, with every stage captured in a NEW transcript under transcripts/audit/phase1d_r1/ (final_regression_python.txt, final_regression_kdemos.txt, final_regression_lc_1d.txt, final_regression_kevm_lin.txt). Historical transcripts were never overwritten; where a stage had a frozen record, the new run is compared against that record explicitly inside the new transcript.

Stage	Required	Result
Phase-0 Python baseline	15/15	15/15 (unittest, exit 0)
Phase-0 abstract demos	13/13	13/13; verdict sequence byte-identical to frozen v5 [2]
Phase-1B generalized (hs)	PASS	10/10 PASS
Phase-1B generalized (llvm)	PASS	10/10 PASS
Phase-1C krypto probe	PASS	8/8 ok=true (live shim rebuild)
Phase-1C ck demos	PASS	commit / reject / commit
Phase-1D positive	PASS	6/6 lin demos, verdicts identical to the frozen record
L7 permanent suite	10/10	10/10 (L7-A...L7-J)
Comparison audit	33/33	33 probes, zero demonstrated deviations
Hostile mutation harness	21/21 rejected	21/21 rejected; UNRESOLVED marker absent
K composition	1 positive + 3 negatives	pos=valid; neg1=neg2=neg3=invalid-state-composition (exit 0, two independent runs)
KEVM lineage positive	COMMIT	lin_evm_multi: record 0 committed, n=1, head 899b6716... byte-identical to the frozen post-fix value
KEVM lineage negative	REJECT	lin_evm_overflow: 0 records, storages restored (atomicity)

The known pre-fix L7 attack remains preserved as historical evidence:

python-gen/test_lin_attack_L7.py (whose header documents that its T2 expectation describes the PRE-FIX verifier), the frozen transcript l7_attack_prefix.txt, and the permanent post-fix suite test_lin_verify_L7.py whose L7-B class is that same attack, now rejected. Two environmental recoveries were made during this stage and are recorded in the scripts: the stddef.h header package gap in rebuild_env_1d.sh, and a set -u bash-compatibility fix in run_gen_demos.sh (a split local declaration; behavior identical).

6. Cross-layer byte consistency (mandate Part 8)

The three-layer byte-consistency program was re-run and extended, captured in transcripts/audit/phase1d_r1/final_crosslayer_bytes.txt with 17 of 17 byte-equality checks matching. Section A (Python ↔ K, 9/9) parses the K records from a fresh abstract-demo dump at lin_true_effects record 1 ($Q=\{1:\{0:25\}\}$, $E=\{1:\{0:25\}, 2:\{1:7\}\}$, $P=\{1:\{0:25\}, 2:\{0:50, 1:7\}\}$) and recomputes every artifact with the independent Python verifier: input digest, effect digest, state digest, authority (last 20 bytes of keccak of the public key), the RFC6979 evidence signature over the intent hash, the record signature over the core hash, the child commitment over the canonical full encoding, plus two recovery cross-checks (recover the authority from the K-side evidence and signature bytes). Section B (K ↔ KEVM, 8/8) parses the lin_evm_multi record built over REAL EVM storage diffs by the same frozen K modules, parses the final <storage> cells from the dump, and

reproduces every artifact byte-exactly, including the canonical encoding of the three-account empty pre-snapshot and the storage-diff effect digest.

Canonical serialization, input digest, effect digest, state digest, evidence, signature and child commitment — the seven artifact classes named in the mandate — are all covered on both comparison axes, with authority and recovery checks as additions. One parsing subtlety is documented in the transcript's provenance: K's pretty-printer emits `\f` (formfeed) escapes inside Bytes literals, and the comparison harness escapes them correctly, which matters because a naive parser silently turns `\f` into the two bytes `\` and `f` and spoils equality checks. No claim of historical-execution authentication is made or implied anywhere in this program: these comparisons bind presented artifacts, not the provenance of the execution that produced them.

7. Authorized-dishonest producer (mandate Part 9, core security test)

The core adversarial test was re-run end-to-end and captured in `transcripts/audit/phase1d_r1/final_part9_threat.txt`. The producer holds the valid private key SK1 (secp256k1, the pinned libsecp256k1 path), valid standard keccak, and the ability to recompute every digest; it chooses the presented pre-state/effects/post-state freely and re-signs everything. The attempt constructs the extra-key forgery — the honest post of the scenario plus entry `(2,9):=999` — and builds a record in which `inputD`, `effectD`, `stateD`, the evidence, the signature, and the child commitment are all genuinely computed over the forged artifacts, so every cryptographic field is valid and only the state correspondence is false.

The independent Python verifier rejects the record with first-fail verdict L7-STATE: layers L1 through L6 pass because the producer's cryptography is perfect, and the composition layer is exactly where the forgery dies. The K verifier rejects the same construction — demonstrated with genuine-crypto records built by `LinBuildRec` in the Part 2 final suite — at the additive verdict `invalid-state-composition`, and the 21-class mutation harness independently re-confirms the same layer for class m20, which is this attack. This remains a concrete demonstration inside the modeled verifier domain, not a general cryptographic theorem; the security assumptions it rests on are exactly the ASSUMED items of the evidence classification (collision resistance of keccak256, unforgeability of secp256k1 signatures without the key, RFC6979 nonce determinism across builds).

8. Final git provenance (mandate Parts 4-5)

The final R1 source tree is committed as a descendant of the audited R1 head 5183a44 on branch `phase1c`, with nothing on `main`, `kevm-changes`, or `phase1-start` rewritten. The commit is marked by BOTH a branch and a tag named `phase1d-r1-complete`, and the tree contains every R1 source, proof, test, script, and transcript change that constitutes the final artifact, plus this report. The commit hash itself, its parent hash, and the bundle SHA-256 are recorded in the portable `MANIFEST.txt` and the bundle sidecar rather than inside the commit, for the same self-reference reason that governs the manifest: a document cannot contain a hash of itself.

The fresh bundle `git/srw3-work-r1-final.bundle` is created with `git bundle create ... --all` and therefore contains at least the four required refs — `main`, `kevm-changes`, `phase1-start`, and `phase1d-r1-complete` — together with `phase1c`. It is verified twice: `git bundle verify` inside the main repository, and a full reconstruction in a fresh empty repository that clones the bundle, checks out `phase1d-r1-complete`, and confirms that the reconstructed tree's git archive SHA-256 equals the value recorded for the final commit. The verification transcript, including the ref list and the tree-hash equality, is captured at `transcripts/provenance/git_bundle_r1_final_verify.txt`.

9. Portable manifest repair (mandate Part 6)

The manifest defect was structural: the packaging script created `MANIFEST.txt` as a file in the staging tree and then checksummed all files matching its extension patterns — including `MANIFEST.txt` itself — appending a hash of the file's own earlier state, a value that could never re-verify. The repaired packager generates the manifest from all artifact files EXCEPT the manifest itself (the exclusion is explicit in the file-list construction), then computes the SHA-256 of the complete ZIP into a separate sidecar file `SRW3-Phase1D-R1-FINAL-Artifact-Bundle.zip.sha256`, so the ZIP integrity and the file inventory are independently checkable without self-reference.

Portability is enforced as before and re-verified: the manifest contains no agent-local absolute paths (no `/home/z/...` anywhere in `MANIFEST.txt`), all entries are relative to the bundle root, and the packaging script fails hard if the `grep` ever matches. The manifest also records the final commit hash, the bundle SHA-256, and the toolchain versions, so a recipient can chain from the ZIP sidecar to the git bundle to the exact audited tree without trusting any out-of-band statement.

10. Terminology and evidence classification (mandate Part 10)

The report's language discipline is retained verbatim and re-verified by search. The term **presented-effect consistency** is kept everywhere and is NOT replaced by "true execution-effect authentication": §6 of the predecessor report states that L7 establishes exactly the internal consistency of the presented triple and nothing more, and every claim in this closure is of that kind. The binding language used is **cryptographically bound under stated assumptions** — now recorded explicitly in the predecessor report's §15 together with the enumeration of those assumptions — and the phrases "cryptographically secure", "tamper-proof", and "provenance guaranteed" appear nowhere in the report or in this closure.

The five-level evidence vocabulary remains the only evaluative vocabulary in use: PROVED BY K, DEMONSTRATED BY KEVM, ASSUMED, REQUIRES CLIENT/PROTOCOL SUPPORT, and NOT YET MECHANIZED, with VERIFIER-REJECTED for concrete forgery demonstrations. The demonstrated statement is deliberately phrased as "the verifier rejects the demonstrated substitution and replay attacks, including fully re-signed ones" — never "the verifier is safe". Cross-domain replay prevention remains REQUIRES CLIENT/PROTOCOL SUPPORT, the Merkle/MPT authenticated-state-proof interface remains NOT YET MECHANIZED (Phase 1E), and calldata-level input binding remains REQUIRES CLIENT/PROTOCOL SUPPORT.

11. Deliverables (mandate Part 11)

The final deliverables are `SRW3-Phase1D-R1-FINAL-Report.md` and `SRW3-Phase1D-R1-FINAL-Report.pdf` (this document), plus `SRW3-Phase1D-R1-FINAL-Artifact-Bundle.zip` with its `SRW3-Phase1D-R1-FINAL-Artifact-Bundle.zip.sha256` sidecar. The bundle contains: the corrected independent Python verifier and its permanent suites; the K-side verifier and composition definitions (`srw3lin.k`, `srw3lin-verify.k`, `srw3lin-r1.k`, `srw3lin-r1-demo.k`); the K successful composition transcript AND the historical failed composition transcript; the L7 attack reproduction (pre-fix script, frozen transcript, and post-fix permanent suite); the 21-class mutation harness; the crypto shim sources with the multiblock fix and the crypto probe suite; all current phase reports; the complete regression transcripts of this closure; the final git bundle `git/srw3-work-r1-final.bundle`; provenance metadata; and the portable self-excluding manifest.

The preclosure bundle remains available unchanged as `SRW3-Phase1D-R1-preclosure.zip` with its own sidecar, and every superseded evidence file remains inside the final bundle beside its replacement, so the closure's own history is part of the archive rather than being erased by it.

12. Completion criterion (mandate Part 12)

Phase 1D-R1 closes when the five conjuncts are all satisfied, and they are: (1) L7 exact equality — canonical map equality in both implementations, asymmetric loop removed, audit-clean (§4, §5); (2) K successful evidence — the composition suite's four verdicts captured verbatim from a live re-compile and the committed definition (§3); (3) full regression — every required stage green with zero drift against the frozen records (§5); (4) final git provenance — descendant commit, branch and tag phase1d-r1-complete, verified all-ref bundle reconstructing the exact tree (§8); (5) portable manifest — self-excluding inventory, separate ZIP sidecar, no absolute paths (§9).

The scientific result frozen by this closure, with its scope stated precisely:

> Within the modeled verifier domain, a validly authorized but dishonest producer cannot make an internally inconsistent presented post-state pass the verifier's state-composition layer after the L7 repair.

Historical true effects remain an open problem requiring authenticated execution/state evidence (the Merkle/MPT interface, NOT YET MECHANIZED, Phase 1E; chain-level attestation, REQUIRES CLIENT/PROTOCOL SUPPORT). Phase 1E remains closed until this closure is accepted; this document is the closure.

13. Artifact index (closure additions)

Artifact	Path
Preclosure freeze	transcripts/audit/phase1d_r1/part0_final_freeze.txt
K composition FINAL transcript	transcripts/audit/phase1d_r1/part7_k_composition_suite_final.txt
Regression: python stages	transcripts/audit/phase1d_r1/final_regression_python.txt
Regression: K demos	transcripts/audit/phase1d_r1/final_regression_kdemos.txt
Regression: 1C + 1D	transcripts/audit/phase1d_r1/final_regression_1c_1d.txt
Regression: KEVM lineage	transcripts/audit/phase1d_r1/final_regression_kevm_lin.txt
Cross-layer bytes	transcripts/audit/phase1d_r1/final_crosslayer_bytes.txt
Authorized-dishonest re-run	transcripts/audit/phase1d_r1/final_part9_threat.txt
Bundle verification	transcripts/provenance/git_bundle_r1_final_verify.txt
Final git bundle	git/srw3-work-r1-final.bundle (in the artifact bundle)
Portable manifest	MANIFEST.txt (self-excluding) + SRW3-Phase1D-R1-FINAL-Artifact-Bundle.zip.sha256